PIPELINE DE MACHINE LEARNING

Análisis y predicción de “El Sueño en los Primeros Meses de Vida”

Autor: Héctor Raúl Huarcaya Chipana

Curso: Machine Learning (Facultad de Ciencias e Ingeniería)

Unidad: Fundamentos de Machine Learning

Actividad: Challenge 2: Pipeline de Machine Learning

Docente: Yannick Patrick Carrasco Merma

Semestre: 2026-II

Enlace del Repositorio:
https://github.com/hectorhch/Pipeline_ML_sueno_neonatal



ÍNDICE

Resumen Ejecutivo 3
1. Introducción 3
2. Presentación del Caso y Definición del Problema 3
2.1 Caso Seleccionado 3
2.2 Planteamiento del Problema 3
2.3 Pregunta del Problema 3
2.4 Objetivo General 3
2.5 Justificación del Uso de IA y Machine Learning 4
3. Espacio de Estados 4
3.1 Formulación Formal 4
3.2 Estados Definidos 4
3.3 Operadores 5
3.4 Restricciones 5
3.5 Representaciones de la Secuencia 5
4. Pipeline de Razonamiento, Búsqueda y Resultados 5
4.1 Objetivo 5
4.2 Algoritmo Seleccionado: Búsqueda en Anchura (BFS) 5
4.3 Código de Implementación 6
4.4 Verificación de Restricciones 6
4.5 Resultado Esperado 6
5. Pipeline del Modelo de Predicción y Visualización 7
5.1 Tarea de Aprendizaje 7
5.2 Justificación del Modelo 7
5.3 Generación de Datos Sintéticos 7
5.4 Entrenamiento y Evaluación 8
5.5 Visualización de Predicciones 8
5.6 Importancia de las variables 8
5.7 Ejemplo de Predicción Individual 9
6. Integración de GitHub y Google Colab 9
6.1 Dependencias 9
7. Limitaciones y Consideraciones Éticas 9
8. Conclusiones 10
9. Referencias 10

Resumen Ejecutivo

Este informe desarrolla una propuesta académica de Pipeline de Machine Learning aplicada al caso Tipo C: sueño en los recién nacidos. Integra la formulación del problema, la representación del espacio de estados, un pipeline de razonamiento y búsqueda y un pipeline predictivo basado en regresión. GitHub se utiliza como repositorio de código y documentación, mientras que Google Colab es el entorno propuesto para ejecutar los notebooks.

Para demostrar el funcionamiento técnico del modelo predictivo se generan datos sintéticos con variables de entrada como edad en meses, horas de sueño diarias y actividad motora, y una variable objetivo de duración del ciclo de sueño en minutos. Se propone Random Forest Regressor y se evalúa mediante MAE, RMSE y R^2. Los datos son artificiales y no representan observaciones clínicas; por ello, el proyecto demuestra un flujo computacional, no una herramienta diagnóstica.

1. Introducción

El sueño durante los primeros meses de vida puede estudiarse mediante registros de duración, frecuencia y distribución de los periodos de descanso. Analizar varios registros permite explorar patrones y relaciones entre variables; sin embargo, la utilidad de cualquier conclusión depende de la calidad, disponibilidad y representatividad de los datos.

En el marco del Challenge 1, se propone un flujo que integra conceptos de Inteligencia Artificial y Machine Learning con herramientas de desarrollo. El trabajo se organiza en cuatro componentes: presentación del problema, espacio de estados, pipeline de razonamiento y búsqueda, y pipeline del modelo predictivo y visualización.

2. Presentación del Caso y Definición del Problema

2.1 Caso Seleccionado

Caso Tipo C: Sueño en los recién nacidos.

2.2 Planteamiento del Problema

Los patrones de sueño durante los primeros meses de vida pueden variar en duración y distribución. El análisis manual de registros puede dificultar la identificación de tendencias y asociaciones entre variables como edad, duración de los ciclos, actividad motora y horas totales de descanso. Se plantea desarrollar un pipeline de Machine Learning que procese los datos disponibles y estime la duración de un ciclo de sueño a partir de características de entrada.

2.3 Pregunta del Problema

¿De qué manera las técnicas de Machine Learning permiten analizar patrones y estimar la duración de los ciclos de sueño en recién nacidos a partir de variables registradas sobre su descanso?

2.4 Objetivo General

Desarrollar un pipeline reproducible mediante GitHub y Google Colab para analizar datos relacionados con el sueño neonatal y entrenar y evaluar un modelo de regresión que estime la duración de un ciclo de sueño a partir de variables de entrada.

2.5 Justificación del Uso de IA y Machine Learning

- Identificación de patrones: explorar asociaciones entre variables de los registros de sueño.
- Predicción: estimar una variable numérica a partir de ejemplos previos.
- Análisis multivariable: estudiar conjuntamente distintas características registradas.
- Automatización y reproducibilidad: ejecutar procedimientos repetibles y documentar los experimentos.

Machine Learning no garantiza predicciones exactas ni demuestra causalidad; su utilidad debe evaluarse con datos independientes y métricas apropiadas.

3. Espacio de Estados

3.1 Formulación Formal

El problema se representa mediante la tupla:

$$P = (S, O, S_0, Sg, R)$$

S es el conjunto de estados; O, los operadores; S_0, el estado inicial; Sg, el estado objetivo; y R, las restricciones.

3.2 Estados Definidos

Código	Estado	Descripción
S_0	Problema definido	Se delimita la variable que se desea estimar.
S_1	Datos recopilados	Se dispone de registros pertinentes.
S_2	Datos validados	Se verifica la estructura, integridad y calidad.
S_3	Datos preprocesados	Se corrigen faltantes e inconsistencias.
S_4	Variables seleccionadas	Se establecen predictores y variable objetivo.
S_5	Datos divididos	Se separan los conjuntos de entrenamiento y prueba.
S_6	Modelo entrenado	Se ajusta el algoritmo con datos de entrenamiento.
S_7	Modelo evaluado	Se calculan métricas con datos no usados para entrenar.
Sg	Objetivo alcanzado	Modelo evaluado según criterios y resultados documentados.

3.3 Operadores

Operador	Acción	Transición
O_1	Recopilar o cargar datos	$S_0 \rightarrow S_1$
O_2	Validar datos	$S_1 \rightarrow S_2$
O_3	Limpiar y transformar	$S_2 \rightarrow S_3$
O_4	Seleccionar variables	$S_3 \rightarrow S_4$
O_5	Dividir los datos	$S_4 \rightarrow S_5$
O_6	Entrenar el modelo	$S_5 \rightarrow S_6$
O_7	Evaluar predicciones	$S_6 \rightarrow S_7$
O_8	Aceptar y documentar si cumple	$S_7 \rightarrow Sg$
O_9	Ajustar el proceso si no cumple	$S_7 \rightarrow S_3$

3.4 Restricciones

• Los datos deben ser pertinentes y suficientes para la tarea planteada.

• Se deben revisar valores faltantes, duplicados e inconsistencias.

• La variable objetivo debe contener valores numéricos válidos.

• Los datos de prueba deben mantenerse separados del entrenamiento para evitar fuga de información.

• El modelo debe evaluarse mediante métricas definidas y justificadas.

• Los datos personales o clínicos identificables deben protegerse.

• El código, los procedimientos y los resultados deben documentarse para favorecer la reproducibilidad.

3.5 Representaciones de la Secuencia

$$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4 \rightarrow S_5 \rightarrow S_6 \rightarrow S_7 \rightarrow Sg$$

Si la evaluación no cumple los criterios establecidos, el proceso vuelve a una etapa anterior para ajustar datos, variables o algoritmo. El estado objetivo solo se alcanza cuando se satisfacen las condiciones definidas.

4. Pipeline de Razonamiento, Búsqueda y Resultados

4.1 Objetivo

Implementar un notebook que represente las etapas de resolución del problema y explore una ruta desde el estado inicial hasta el objetivo mediante un algoritmo de búsqueda.

4.2 Algoritmo Seleccionado: Búsqueda en Anchura (BFS)

BFS (Breadth-First Search) recorre el grafo por niveles utilizando una cola. En un grafo no ponderado, encuentra una ruta con el menor número de transiciones cuando existe una ruta al objetivo. Aquí se aplica al grafo conceptual de estados; no entrena el modelo predictivo ni descubre patrones clínicos.

4.3 Código de Implementación

```
from collections import deque

estados = {
    "S0": "Problema definido",
    "S1": "Datos recopilados",
    "S2": "Datos validados",
    "S3": "Datos preprocesados",
    "S4": "Variables seleccionadas",
    "S5": "Datos divididos",
    "S6": "Modelo entrenado",
    "S7": "Modelo evaluado",
    "Sg": "Modelo aceptado y documentado"
}

operadores = {
    "S0": [("O1", "S1", "Recopilar datos")],
    "S1": [("O2", "S2", "Validar datos")],
    "S2": [("O3", "S3", "Preprocesar datos")],
    "S3": [("O4", "S4", "Seleccionar variables")],
    "S4": [("O5", "S5", "Dividir datos")],
    "S5": [("O6", "S6", "Entrenar modelo")],
    "S6": [("O7", "S7", "Evaluar modelo")],
    "S7": [("O8", "Sg", "Aceptar y documentar"),
           ("O9", "S3", "Ajustar el proceso")],
    "Sg": []
}

def busqueda_bfs(inicio, objetivo, grafo):
    cola = deque([(inicio, [inicio])])
    visitados = set()
    while cola:
        actual, camino = cola.popleft()
        if actual == objetivo:
            return camino
        if actual in visitados:
            continue
        visitados.add(actual)
        for operador, destino, accion in grafo.get(actual, []):
            if destino not in visitados:
                cola.append((destino, camino + [destino]))
    return None

ruta = busqueda_bfs("S0", "Sg", operadores)
print([estados[s] for s in ruta] if ruta else "Ruta no encontrada")
```

4.4 Verificación de Restricciones

```
restricciones = {
    "datos_disponibles": True,
    "datos_validados": True,
    "variable_objetivo_valida": True,
    "sin_fuga_de_datos": True,
    "privacidad_protegida": True,
    "modelo_evaluable": True
}

cumple = all(restricciones.values())
if ruta and cumple:
    print("Ruta encontrada y restricciones satisfechas.")
else:
    print("La solución requiere revisión.")
```

Los indicadores se inicializan con valores booleanos para ilustrar la lógica. En una implementación real, cada condición debe verificarse a partir del dataset y de los resultados; asignar True no constituye una comprobación.

4.5 Resultado Esperado

La búsqueda devuelve una ruta desde S_0 hasta Sg a través de las transiciones definidas. Esto comprueba el funcionamiento del algoritmo sobre el grafo conceptual, pero no

demuestra que el modelo predictivo sea clínicamente válido ni que haya alcanzado un error específico.

5. Pipeline del Modelo de Predicción y Visualización

5.1 Tarea de Aprendizaje

Elemento	Definición
Tipo de aprendizaje	Supervisado
Tarea	Regresión
Variable objetivo	Duración del ciclo de sueño en minutos
Predictores	Edad en meses, horas de sueño diarias y actividad motora
Modelo	Random Forest Regressor
Evaluación	MAE, RMSE y R^2
Datos	Sintéticos, generados con fines académicos

5.2 Justificación del Modelo

Random Forest Regressor combina predicciones de múltiples árboles de decisión. Puede representar relaciones no lineales y permite estimar la importancia relativa de las características. Se elige como modelo de referencia para una regresión con variables numéricas. Su desempeño debe verificarse con datos de prueba; no se presupone que supere a otros algoritmos.

5.3 Generación de Datos Sintéticos

Se generan 500 registros artificiales. Las variables de entrada son edad en meses, horas de sueño diarias y actividad motora. La duración del ciclo se calcula mediante una relación matemática predefinida y ruido aleatorio. Este mecanismo permite practicar el entrenamiento y la evaluación, pero no representa observaciones de recién nacidos.

```
import numpy as np
import pandas as pd


np.random.seed(42)
n = 500
edad_meses = np.random.uniform(0, 6, n)
horas_sueno_diarias = np.random.uniform(10, 18, n)
actividad_motora = np.random.uniform(0, 10, n)


duracion_ciclo = (
    42 + 3.5 * edad_meses
    + 2.0 * horas_sueno_diarias
    - 0.8 * actividad_motora
    + np.random.normal(0, 5, n)
)
duracion_ciclo = np.clip(duracion_ciclo, 20, 120)


df = pd.DataFrame({
    "edad_meses": edad_meses,
    "horas_sueno_diarias": horas_sueno_diarias,
    "actividad_motora": actividad_motora,
    "duracion_ciclo_min": duracion_ciclo
})
df.head()
```

5.4 Entrenamiento y Evaluación

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

X = df[["edad_meses", "horas_sueno_diarias", "actividad_motora"]]
y = df["duracion_ciclo_min"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

modelo = RandomForestRegressor(
    n_estimators=100, max_depth=8,
    min_samples_leaf=2, random_state=42
)
modelo.fit(X_train, y_train)
y_pred = modelo.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"MAE: {mae:.2f} minutos")
print(f"RMSE: {rmse:.2f} minutos")
print(f"R²: {r2:.3f}")
```

Se destina el 80 % de los registros al entrenamiento y el 20 % a la prueba. MAE expresa el error absoluto promedio; RMSE penaliza con mayor intensidad los errores grandes; R² compara el ajuste con una referencia basada en la media. Las cifras finales deben obtenerse al ejecutar el notebook, por lo que este informe no incluye resultados numéricos inventados.

5.5 Visualización de Predicciones

```
import matplotlib.pyplot as plt

plt.figure(figsize=(7, 6))
plt.scatter(y_test, y_pred, alpha=0.7)
limite_min = min(y_test.min(), y_pred.min())
limite_max = max(y_test.max(), y_pred.max())
plt.plot([limite_min, limite_max],
         [limite_min, limite_max], linestyle="--")
plt.xlabel("Duración real (minutos)")
plt.ylabel("Duración predicha (minutos)")
plt.title("Valores reales frente a predicciones")
plt.tight_layout()
plt.show()
```

Los puntos próximos a la diagonal representan predicciones cercanas a los valores generados; la distancia indica visualmente el error.

5.6 Importancia de las variables

```
importancias = pd.DataFrame({
    "Variable": X.columns,
    "Importancia": modelo.feature_importances_
}).sort_values("Importancia", ascending=False)

plt.figure(figsize=(8, 4))
plt.barh(importancias["Variable"], importancias["Importancia"])
plt.xlabel("Importancia relativa")
plt.title("Importancia de las variables")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

La importancia refleja cómo el modelo utiliza las características dentro de este conjunto sintético. No prueba causalidad ni implica que las mismas variables tengan igual relevancia en datos reales.

5.7 Ejemplo de Predicción Individual

```
nuevo_registro = pd.DataFrame({
    "edad_meses": [2.0],
    "horas_sueno_diarias": [14.0],
    "actividad_motora": [4.0]
})
prediccion = modelo.predict(nuevo_registro)[0]
print(f"Duración estimada: {prediccion:.2f} minutos")
```

La predicción ilustra el uso del modelo entrenado. Al proceder de datos sintéticos, no debe utilizarse para orientar decisiones sobre la salud infantil.

6. Integración de GitHub y Google Colab

Herramienta	Función en el proyecto
GitHub	Repositorio de código, control de versiones, documentación y resultados.
Google Colab	Ejecución de notebooks, análisis de datos, entrenamiento y evaluación.
Python	Implementación del flujo de razonamiento y del modelo.
pandas y NumPy	Manipulación de tablas y operaciones numéricas.
scikit-learn	División de datos, entrenamiento y métricas.
Matplotlib	Gráficos exploratorios y visualización de predicciones.

6.1 Dependencias

numpy

pandas

matplotlib

scikit-learn

7. Limitaciones y Consideraciones Éticas

• Los datos son sintéticos y no permiten validar el desempeño en una población real.

• La relación matemática utilizada para generar la duración es artificial; el modelo aprende patrones de esa simulación.

• BFS demuestra una ruta en un grafo definido manualmente y no aprende de datos.

• Las métricas deben calcularse al ejecutar el notebook y no reemplazarse por cifras no verificadas.

• Si se incorporan datos reales de bebés, se requieren autorización, protección de privacidad y eliminación de identificadores personales.

• El modelo no es un instrumento diagnóstico y no debe utilizarse para tomar decisiones clínicas.

8. Conclusiones

1. Se formuló un problema de Machine Learning relacionado con la estimación de la duración de ciclos de sueño neonatal.

2. El espacio de estados organizó el proceso en etapas, operadores y restricciones, desde la definición del problema hasta la documentación del modelo evaluado.

3. BFS permite representar y recorrer una ruta de solución dentro del grafo conceptual del pipeline.

4. El pipeline predictivo implementa una tarea de regresión supervisada con Random Forest Regressor, métricas de evaluación y visualizaciones interpretativas.

5. La integración de GitHub y Google Colab favorece la organización, ejecución y reproducibilidad del trabajo.

6. Los resultados son demostrativos porque se generan con datos sintéticos y no constituyen evidencia clínica.

9. Referencias

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Scikit-learn developers. RandomForestRegressor documentation. https://scikitlearn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html

Scikit-learn developers. Model evaluation: quantifying the quality of predictions. https://scikitlearn.org/stable/modules/model_evaluation.html

GitHub Docs. About writing and formatting on GitHub. https://docs.github.com/en/get-started/writing-ongithub

Google Colab. Welcome to Colab. https://colab.research.google.com/